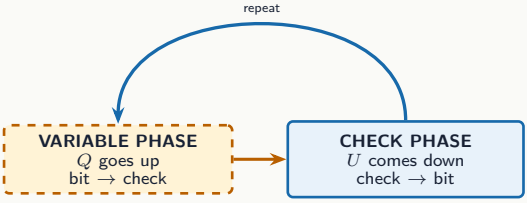
Beat 1 of 7: Two loops on one Tanner graph

Belief propagation (BP)

retains fixed $L[3]$ + current $U[4]$

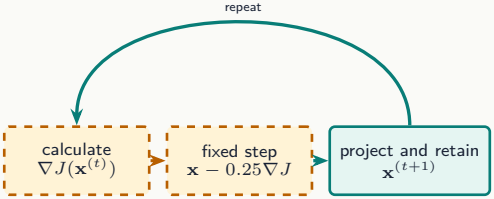


$$Q_{i\to c}^{(t)} = L_i + \sum_{d\in N(i)\setminus\{c\}} U_{d\to i}^{(t-1)}$$

Each outgoing value retains a recipient address.

Projected GD

retains fixed $\ell[3]$ + current $\mathbf{x}[3]$



$$\mathbf{x}^{(t+1)} = \Pi_{[-1,1]^3}\left(\mathbf{x}^{(t)} - 0.25\nabla J(\mathbf{x}^{(t)})\right)$$

$$J(\mathbf{x}) = -\tfrac{1}{2}\boldsymbol{\ell}^{\mathsf{T}}\mathbf{x} + 2\left[(1-x_1x_2)^2 + (1-x_2x_3)^2\right]$$

WHAT DIFFERS HERE? Both methods use the graph, but BP advances edge-addressed messages while projected GD updates one scalar per bit.

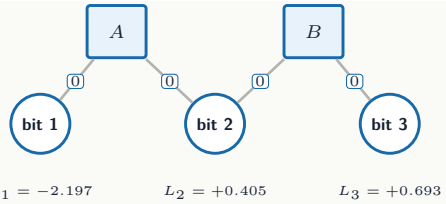
Exact sum-product BP vs constructed fixed-step projected-GD teaching control; synchronized beats do not imply equal arithmetic, memory, objectives, complexity, or iteration counts

Beat 2 of 7: Store L_1, L_2, L_3; initialize all $U^{(0)}_{c\to i} = 0$

Belief propagation (BP)

four directed edge values start at zero

STATE ENTERING BEAT $\mathbf{L} = (-2.197, +0.405, +0.693)$; $\mathbf{U}^{(0)} = (0, 0, 0, 0)$



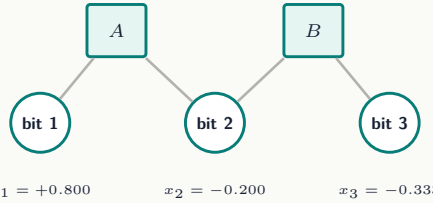
Stored before BP iteration 1: $L[3] + U^{(0)}[4] = 7$ **values.**

Projected GD

one relaxed scalar is initialized per bit

STATE ENTERING BEAT $\boldsymbol{\ell} = -\mathbf{L}$; $\mathbf{x}^{(0)} = \tanh(\boldsymbol{\ell}/2)$

$\boldsymbol{\ell} = (+2.197, -0.405, -0.693)$



Stored before the first GD update: $\boldsymbol{\ell}[3] + \mathbf{x}^{(0)}[3] = 6$ **values.**

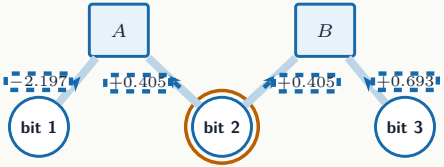
WHAT DIFFERS HERE? The declared sign relation is $\boldsymbol{\ell} = -\mathbf{L}$. BP initializes directed edge state; projected GD initializes one scalar per bit.

Beat 3 of 7: Variable-node stage forms $Q^{(1)}_{i\to c}$ upward

Belief propagation (BP)

forms four addressed $Q^{(1)}$ messages

STATE ENTERING BEAT $\mathbf{U}^{(0)} = (0, 0, 0, 0)$ remains frozen during this variable phase

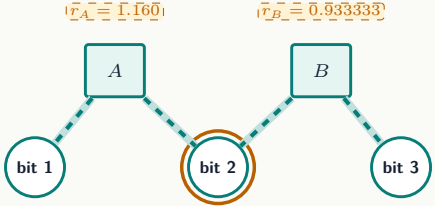


$$\mathbf{Q}^{(1)} = (-2.197,\ +0.405,\ +0.405,\ +0.693)$$

Projected GD

calculates two temporary check residuals

STATE ENTERING BEAT $\mathbf{x}^{(0)} = (+0.800, -0.200, -0.333)$ remains frozen



$$r_c = 1 - \prod_{j \in c} x_j, \qquad (r_A, r_B) = (1.160, 0.933333)$$

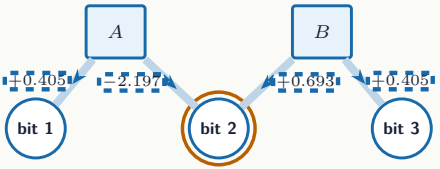
WHAT DIFFERS HERE? BP arrowheads show values travelling to named recipients. GD dashed lines show mathematical dependence, not travelling messages.

Beat 4 of 7: Check-node stage forms $U_{c\to i}^{(1)}$ downward

Belief propagation (BP)

new $U^{(1)}$ messages become directed state

STATE ENTERING BEAT each degree-two check reads its pair of $Q^{(1)}$ messages

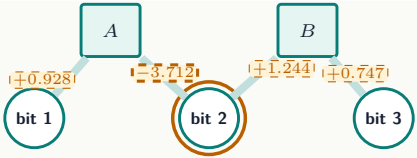


$$\mathbf{U}^{(1)} = (+0.405, -2.197, +0.693, +0.405)$$

Projected GD

four temporary terms contribute to the gradient

STATE ENTERING BEAT all $g_{c,i}$ use the frozen $\mathbf{x}^{(0)}$ and residual r_c



$$g_{c,i} = -4r_c \prod_{\substack{j\in c \\ j\neq i}} x_j$$

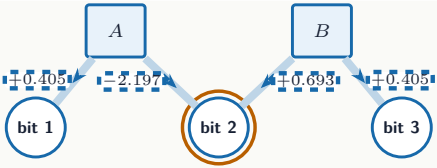
WHAT DIFFERS HERE? BP retains four directed messages. In GD, $g_{A,1}$ contributes to $\partial J/\partial x_1$; $g_{A,2}$ and $g_{B,2}$ contribute to $\partial J/\partial x_2$; and $g_{B,3}$ contributes to $\partial J/\partial x_3$.

Beat 4 continued: Compute $g_{A,2}$

Belief propagation (BP)

the completed check-to-variable messages remain stored

STATE ENTERING BEAT BP does not advance on this continuation slide

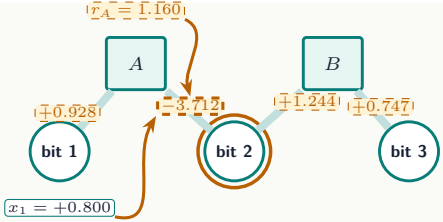


$$\mathbf{U}^{(1)} = (+0.405, -2.197, +0.693, +0.405)$$

Projected GD

specialize the general rule to one temporary term

STATE ENTERING BEAT use the frozen $x_1 = +0.800$, $x_2 = -0.200$, and $r_A = 1.160$



$$g_{c,i} = -4r_c \prod_{\substack{j \in c \\ j \neq i}} x_j \implies c = A,\ i = 2,\ A = \{1, 2\}$$

$$r_A = 1 - x_1 x_2 = 1 - (0.800)(-0.200) = 1.160$$

$$g_{A,2} = -4 r_A x_1 = -4(1.160)(0.800) = \mathbf{-3.712}$$

Belief propagation (BP)

bit 2 prepares a different value for each check

STATE ENTERING BEAT $L_2 = +0.405$, $U^{(1)}_{A\to 2} = -2.197$, $U^{(1)}_{B\to 2} = +0.693$

Send to A: leave out A's preceding contribution

$$Q^{(2)}_{2\to A} = L_2 + U^{(1)}_{B\to 2} = 0.405 + 0.693 = +\mathbf{1.099}$$

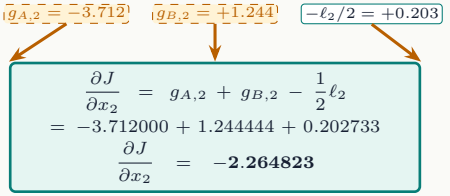
Send to B: leave out B's preceding contribution

$$Q^{(2)}_{2\to B} = L_2 + U^{(1)}_{A\to 2} = 0.405 - 2.197 = -\mathbf{1.792}$$

Projected GD

three terms form $\partial J/\partial x_2$

STATE ENTERING BEAT $g_{A,2} = -3.712$, $g_{B,2} = +1.244$, $-\ell_2/2 = +0.203$



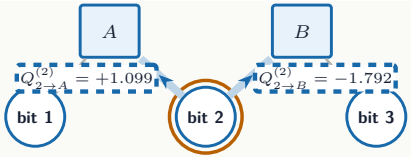
$\partial J/\partial x_2$ is one number used to update x_2; GD does not form separate values addressed to A and B.

WHAT DIFFERS HERE? For bit 2, BP forms $Q^{(2)}_{2\to A}$ and $Q^{(2)}_{2\to B}$ separately. GD sums $g_{A,2}$, $g_{B,2}$, and $-\ell_2/2$ to form $\partial J/\partial x_2$, which is used to update x_2.

Belief propagation (BP)

bit 2 presents two recipient-specific values

STATE ENTERING BEAT the second variable phase has formed $\mathbf{Q}^{(2)}$

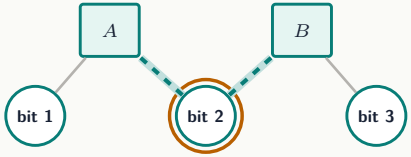


$$\mathbf{Q}^{(2)} = (-2.197, +1.099, -1.792, +0.693)$$

Projected GD

both checks read one updated bit-2 value

STATE ENTERING BEAT $\nabla J(\mathbf{x}^{(0)}) = (-0.171, -2.265, +1.093)$



$x_1^{(1)} = +0.843$ $x_2^{(1)} = +0.366$ $x_3^{(1)} = -0.607$

$$\mathbf{z}^{(0)} = \mathbf{x}^{(0)} - 0.25\nabla J = (+0.843, +0.366, -0.607),$$
$$\mathbf{x}^{(1)} = \Pi_{[-1,1]^3}(\mathbf{z}^{(0)}) = \mathbf{z}^{(0)}.$$

WHAT DIFFERS HERE? This is the visible state boundary: BP has two addressed bit-2 values; projected GD retains one bit-2 scalar.

Beat 7 of 7: What crosses the update boundary

Belief propagation (BP)

boundary state: seven numerical values

STATE ENTERING BEAT fixed $L[3]$ plus current directed $U^{(t)}[4]$

$$L[3] + U^{(t)}[4] = 7$$

Four separately addressed check-to-variable messages remain. Q and Λ can be recalculated.

Implementation note: $L[3]$ plus two four-position arrays allocates 11 positions; one degree-two conversion temporary gives peak 12.

Projected GD

boundary state: six numerical values

STATE ENTERING BEAT fixed $\boldsymbol{\ell}[3]$ plus current relaxed vector $\mathbf{x}^{(t)}[3]$

$$\boldsymbol{\ell}[3] + \mathbf{x}^{(t)}[3] = 6$$

Three current scalars remain, one for each bit. Residuals, $g_{c,i}$, ∇J, and the trial point can be recalculated.

Working-storage peak depends on implementation and is not included in the count of six.

WHAT DIFFERS HERE? The deck isolates a mechanism difference in representation, addressing, retention, and update. It does not establish which decoder performs better.

From the state boundary to a causal test

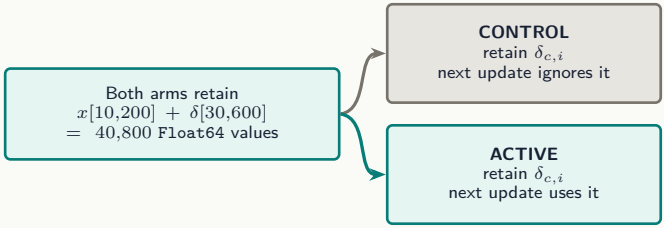
RQ2 QUESTION If both projected-GD arms retain an offset $\delta_{c,i}$ for every Tanner-graph edge, does allowing the next update to *use* those offsets change whether the hard decision equals the transmitted word?

WHAT THE TUTORIAL SHOWED

BP retains directed variable-to-check and check-to-variable messages. The teaching projected-GD example retains one scalar per bit and recalculates terms from the parity checks.

That observation alone did not establish a performance consequence.

HELD FIXED WITHIN EACH PAIR regular $(3, 6)$ graph, transmitted word, received samples and channel LLRs, initialization, objective weights, preconditioner, backtracking procedure, 30-update budget, and hard-decision rule.



This is a projected-GD bridge ablation study. The offsets are not BP messages, and neither arm calls BP.

The controlled fork occurs at update 2

Entering update 2, all 200 pairs have exactly equal $\mathbf{x}^{(1)}$ and retained $\boldsymbol{\delta}^{(1)}$.

CONTROL: RETAIN, DO NOT USE

Each check reads

$$s_{c,i} = x_i,$$

$$P_0(\mathbf{x}) = \sum_c \left(1 - \prod_{i \in c} x_i\right)^2.$$

The scalar update is clipped to

$$x_i^+ \in [-1, 1].$$

ACTIVE: RETAIN AND USE

Check c reads its own shifted value

$$s_{c,i} = x_i + \delta_{c,i},$$

$$P_\delta(\mathbf{x}) = \sum_c \left(1 - \prod_{i \in c} (x_i + \delta_{c,i})\right)^2.$$

The bounds used by the projection keep every check reading in $[-1, 1]$:

$$x_i^+ \in \left[\max_{c \ni i}(-1 - \delta_{c,i}),\ \min_{c \ni i}(1 - \delta_{c,i})\right].$$

ONE REGISTERED TREATMENT, TWO LINKED EFFECTS: the active arm changes both the parity calculation and the bounds used by the projection.

The trajectories separate at the intended boundary

Paired cases with exactly equal retained hashes

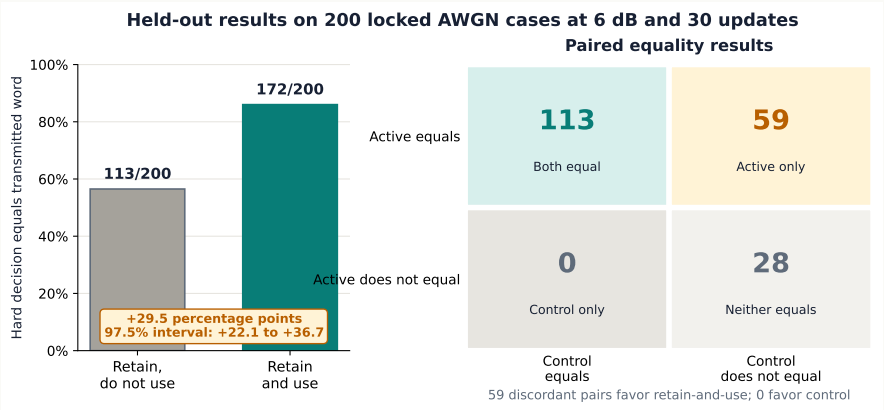
	Update 1	Update 2	Every update 3--30
Scalar state entering	200/200 identical	200/200 identical	0/200 identical
Offsets entering	200/200 identical	200/200 identical	0/200 identical
Gradient vector	200/200 identical	0/200 identical	0/200 identical
Accepted scalar state	200/200 identical	0/200 identical	0/200 identical
Refreshed offsets	200/200 identical	0/200 identical	0/200 identical

At update 2 the entering scalar state and offsets still match; the gradient vector and accepted state do not.

Update 1: all displayed quantities agree in 200/200 pairs. **Update 2:** the entering state still agrees, but 0/200 gradient vectors and accepted scalar states agree. The accepted step size remains equal in every pair, so it is not the first divergence.

Exact hash equality locates the first divergence; it does not measure the magnitude or desirability of that divergence.

Using the retained offsets changes the result

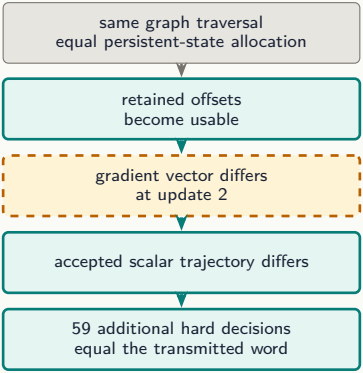


Paired evidence: exact McNemar $p = 3.47 \times 10^{-18}$; Holm-adjusted $p = 6.94 \times 10^{-18}$. Whether the hard decision equals the transmitted word and whether it satisfies every parity check have the same paired table here; neither arm returns a wrong valid word.

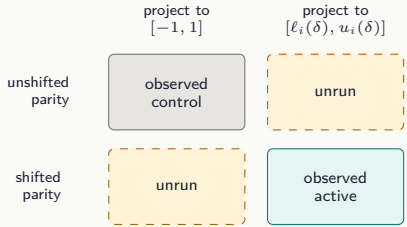
Scope: one regular $(3, 6)$ graph, 200 frozen held-out AWGN cases at physical $E_s/N_0 = 6$ dB, all-zero transmitted word, and 30 accepted updates. This is not a BER-versus-SNR curve or a universal decoder ranking.

What this contributes—and the next causal split

Both arms traverse the same Tanner graph and allocate the same persistent-state shape. Making the retained offsets usable changes the gradient vector, then the accepted scalar trajectory, then the final hard decision.



NEXT: SPLIT THE BUNDLED TREATMENT



Only the diagonal cells have results. The off-diagonal cells are the next proposed experiment.

Not established: that $\delta_{c,i}$ are BP messages, that retention alone caused the gain, that this reproduces BP, or that it uniquely explains the complete native BP–projected-GD gap.